# Model Limits and Decision Context

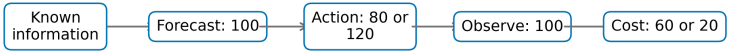
A fitted forecast becomes useful through a specified decision problem.

1. Lecture 4 fitted and compared prediction rules.
2. Now: check limits, quantify consequences, choose feasible actions.
3. Keep information order, assumptions and measured results visible.

# Separate the forecast from the action

Forecast: estimate an unknown outcome. Action: choose before observing it. Loss: evaluate the later consequence. Shortage is demand above stock; excess is stock above demand.

1. Hold the point forecast and realized demand at 100.
2. Stock 80 creates shortage 20; stock 120 creates excess 20.
3. At costs 3 and 1, realized losses are 60 and 20.

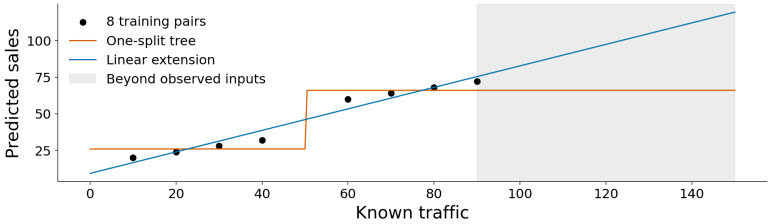



Coding Note, Example 5.1.

# A tree returns a stored leaf mean

Extrapolation uses input conditions beyond those represented in training.

1. Fit the same eight traffic/sales pairs as Lecture 4.
2. Query beyond maximum observed traffic 90.
3. At traffic 140 the tree still returns stored leaf mean 66.

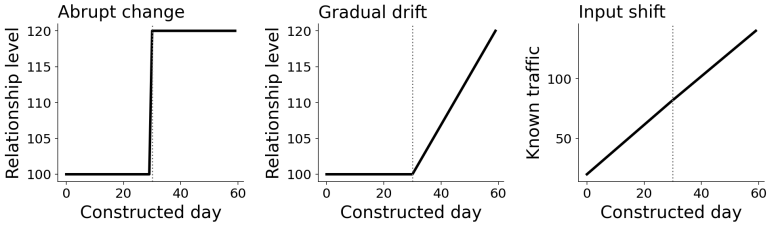



Coding Note, Example 5.2.

# Distinguish three ways the world can change

Regime change is abrupt; drift is gradual; inputs can shift while the outcome rule stays fixed.

1. Construct an abrupt level jump of 20 starting at day 30.
2. Contrast a gradual change over the later dates.
3. In the third panel, change inputs while retaining the rule.

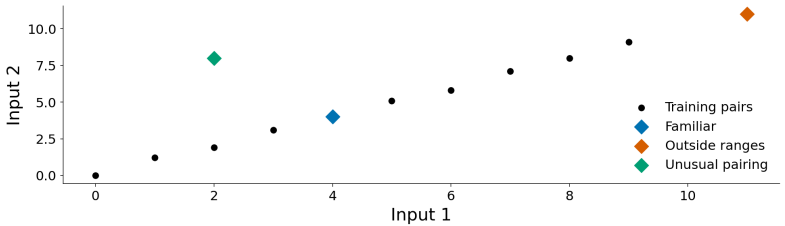



Coding Note, Example 5.3.

# Check input ranges and observed pairings

Training support describes observed input conditions, including which values occurred together.

1. Training pairs lie close to a diagonal band.
2. The query (2,8) passes both coordinate ranges.
3. Its unfamiliar pairing remains visible on the joint plot.

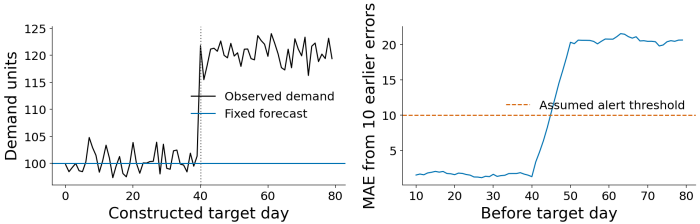



Coding Note, Example 5.4.

# Monitor errors only after outcomes arrive

Monitoring uses completed outcomes. An alert flags a specified condition for review.

1. Before each target, use the previous 10 completed errors.
2. Shift before rolling; compare past MAE with threshold 10.
3. The simulated day-40 change triggers the first alert on day 45.

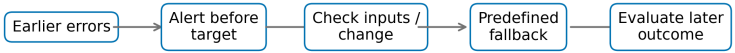



Coding Note, Example 5.5.

# Connect an alert to an explicit fallback rule

A fallback is a predefined response. Capacity states the largest quantity available under the plan.

1. A prior-error signal selects the ordinary or review procedure.
2. The stated plan uses stock 100 or 120 before the target.
3. Observe the outcome later and score the chosen action.



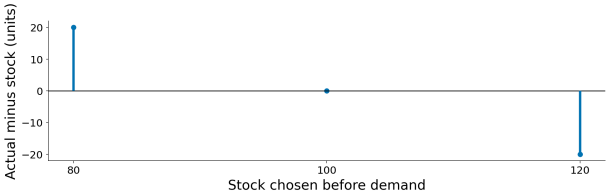

Coding Note, Example 5.6.

# Keep error direction and business units explicit

$$e = y - a$$

Shortage means demand above stock; excess means stock above demand.

1. For actual demand 100, compare stock 80,100,120.
2. Actual minus stock gives +20,0,-20.
3. Keep shortage and excess quantities nonnegative.



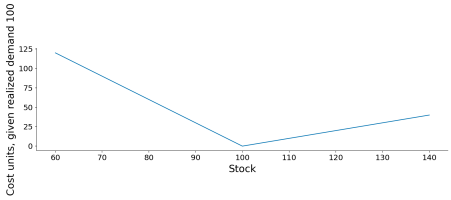

Coding Note, Example 5.7.

# Calculate asymmetric realized loss

$$L(y, a) = c_u \max(y - a, 0) + c_o \max(a - y, 0)$$

Asymmetric loss gives the two error directions different costs per unit.

1. Shortage: multiply missing units by 3.
2. Excess: multiply remaining units by 1.
3. Add the blocks; observed demand 100 gives costs 60,0,20.



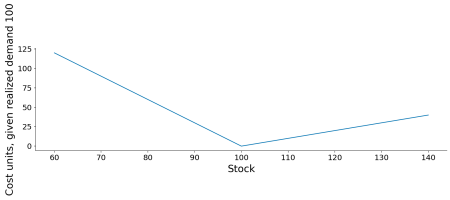

Coding Note, Example 5.8.

# Read the loss as two cost blocks

$$L(y, a) = c_u \max(y - a, 0) + c_o \max(a - y, 0)$$

Each positive-part block counts one mistake direction.

1. At demand 100 and stock 80, shortage is 20 and excess is zero.
2. Multiply shortage by 3 to obtain cost 60.
3. At stock 120, the excess block gives 20 times 1, or cost 20.



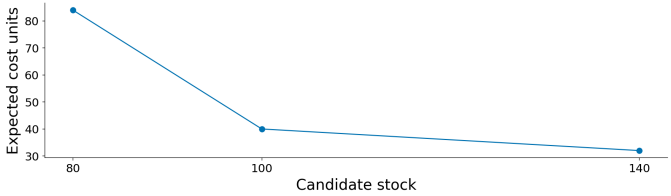

Coding Note, Example 5.8.

# Choose using expected loss before seeing demand

$$\mathbb{E}[L(Y, a)] = \sum_j p_j L(y_j, a)$$

Expected loss is a probability-weighted average over possible outcomes; probabilities sum to one.

1. Demand cases 80,100,140 have assumed weights .2,.5,.3.
2. Evaluate every action against every case.
3. Stock 80,100,140 gives expected cost 84,40,32.

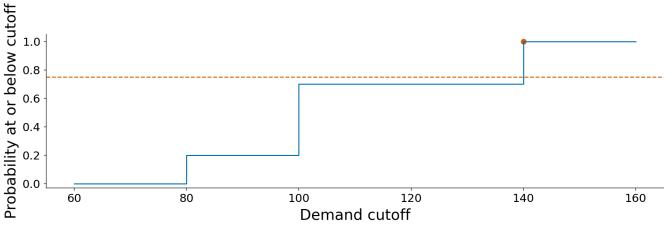



Coding Note, Example 5.9.

# Read a quantile from cumulative probability

A cumulative probability adds mass at or below a cutoff. A quantile is a cutoff reaching a specified probability.

1. Cumulative masses at 80,100,140 are .2,.7,1.0.
2. The median cutoff is 100.
3. The first demand cutoff reaching .75 is 140.



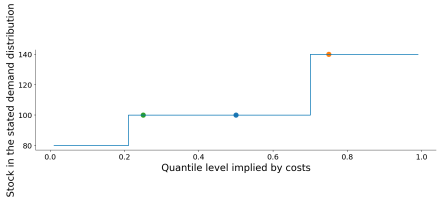

Coding Note, Example 5.10.

# Derive the cost-implied quantile direction

$$\tau^* = \frac{c_u}{c_u + c_o}$$

The cost-implied quantile balances the two directions of marginal loss.

1. The numerator is shortage cost.
2. Shortage:excess costs 3:1 imply .75; costs 1:3 imply .25.
3. Read a demand quantity from that level; keep probability and action distinct.

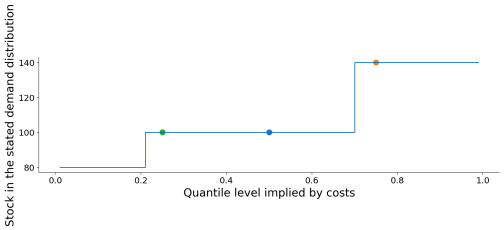



Coding Note, Example 5.11.

# Why this cost belongs in the numerator

A marginal effect is the change from adding a small amount of stock.

1. If demand exceeds stock, the addition can save shortage cost.
2. If stock already exceeds demand, it creates excess cost.
3. Balancing expected marginal effects selects the cost-implied quantile.

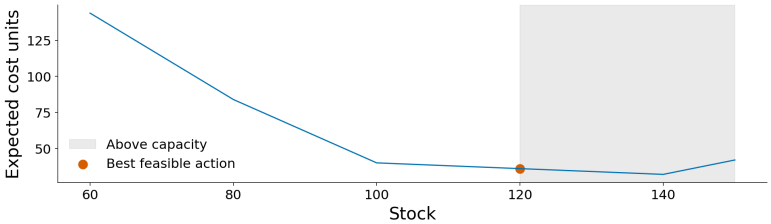



Coding Note, Example 5.11.

# Impose a feasible action set

A constraint limits which actions are feasible; optimize over that set.

1. The unconstrained .75-quantile action is 140.
2. Available capacity is 120 units.
3. The best feasible action is 120, expected cost 36.

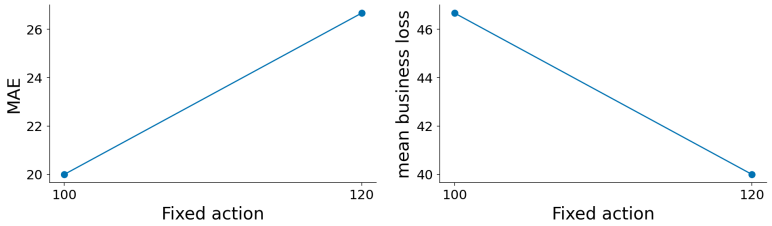



Coding Note, Example 5.12.

# Separate prediction accuracy from decision loss

MAE scores closeness. Business loss scores the consequence of the action.

1. Here each rule stocks its constant point forecast.
2. On the same three cases, MAE prefers 100: 20 versus 26.67.
3. The 3:1 business loss prefers 120: 40 versus 46.67.

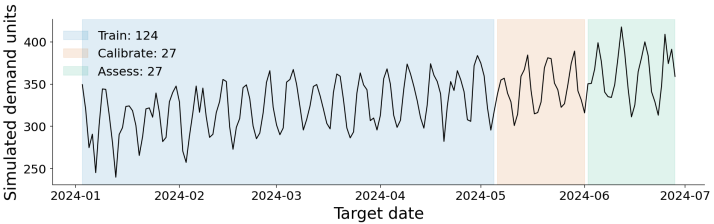



Coding Note, Example 5.13.

# Reuse the classroom data with an honest chronology

Calibration uses errors of earlier saved forecasts from the same unchanged procedure.

1. Reuse seed-417 demand and two known lag inputs.
2. Split 178 usable rows into 124 train,27 calibrate,27 assess.
3. Fit training only; keep the model fixed after calibrating its errors.

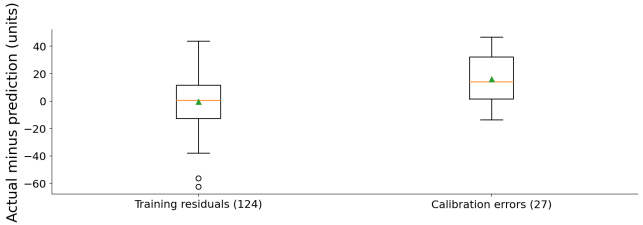



Coding Note, Example 5.14.

# Distinguish training residuals from later forecast errors

Training residuals describe fitted outcomes; later forecast errors compare saved predictions with subsequent outcomes.

1. Use the specified forest on 124 training observations.
2. Measured training MAE: 14.65; calibration MAE: 18.31.
3. Reassess later errors when inputs or the outcome relationship change.



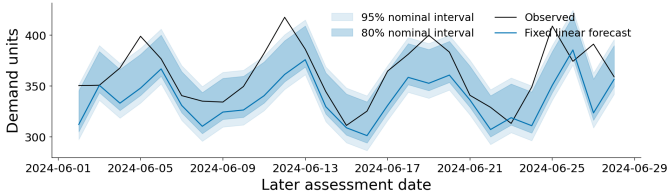

Coding Note, Example 5.15.

# Construct an empirical prediction interval

$$[\ell_t, u_t] = [\hat{y}_{t|t-1} + \hat{q}_{\alpha/2},\ \hat{y}_{t|t-1} + \hat{q}_{1-\alpha/2}]$$

A prediction interval bounds future outcomes. Its nominal coverage is a target, to be checked on outcomes.

1. Read .10/.90 offsets from 27 calibration errors.
2. Add -7.18 and +33.21 to each saved point forecast.
3. Fixed offsets give every 80% band the same width, 40.39.




Coding Note, Example 5.16.

# Fixed error offsets give fixed width

- Four saved later forecasts from the same fixed procedure.
- The same calibration offsets are added on every date.
- Width remains 40.39 demand units.

| Point | Lower 80% | Upper 80% | Width |
|---|---|---|---|
| 312.20 | 305.02 | 345.41 | 40.39 |
| 350.72 | 343.54 | 383.92 | 40.39 |
| 333.15 | 325.97 | 366.36 | 40.39 |
| 347.94 | 340.76 | 381.15 | 40.39 |

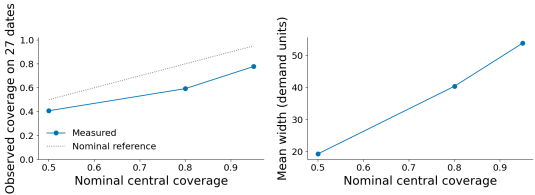
Coding Note, Example 5.16.

# Measure coverage and width together

$$\widehat{\text{coverage}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\{\ell_i \le y_i \le u_i\}$$

Coverage: fraction inside saved bounds. Width: upper minus lower, in outcome units.

1. Use identical later dates for nominal 50%,80%,95% intervals.
2. Observed coverage is 40.74%,59.26%,77.78% on 27 cases.
3. Report widths 19.31,40.39,53.82 beside those fractions.

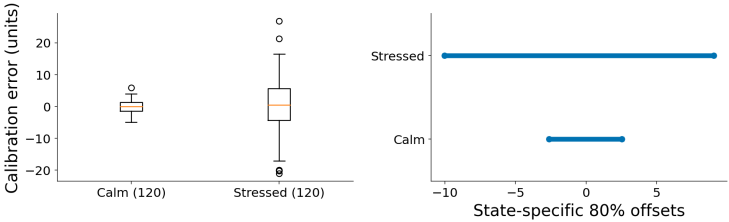



Coding Note, Example 5.17.

# Make state-dependent width an explicit method

A known state can change error spread. Pooled offsets combine states; state-specific offsets separate them. Standard deviation measures spread in error units.

1. Separate simulation: calm/stressed error standard deviations 2 and 8.
2. Use 120 calibration errors per state, then 60 later errors each.
3. State-specific 80% widths are 5.15 and 19.05.




Coding Note, Example 5.18.

# Check coverage within the known state

Compare pooled and state-specific coverage within each known state.

| State | Method | Coverage | Width |
|---|---|---|---|
| Calm | Pooled | 100.00% | 11.47 |
| Calm | State-specific | 90.00% | 5.15 |
| Stressed | Pooled | 41.67% | 11.47 |
| Stressed | State-specific | 76.67% | 19.05 |

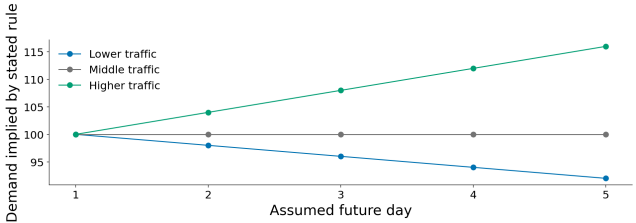
- Each row uses 60 later simulated outcomes in its state.
- Compare each observed fraction with the 80% nominal target.

Coding Note, Example 5.18.

# Trace scenario inputs through a forecasting rule

A scenario specifies future inputs and evaluates the rule under those assumptions.

1. Assumed rule: baseline 80 plus twice traffic, promotion off.
2. Feed three stated traffic paths through that same rule.
3. Day-five demand is 92,100,116 under the three stated traffic paths.

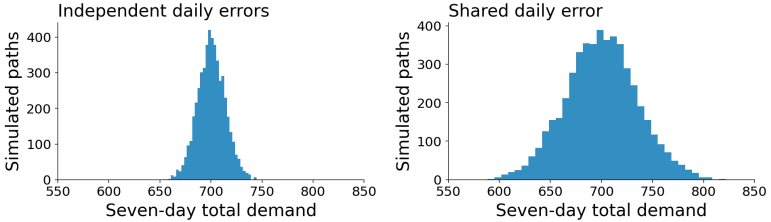



Coding Note, Example 5.19.

# Build weekly intervals from complete daily paths

A joint distribution describes outcomes together. Dependence means that learning one outcome can change the distribution of another.

1. Construct 5000 seven-day paths in each environment.
2. Keep daily mean 100 and error standard deviation 5.
3. Sum within each path, then take quantiles of the totals.

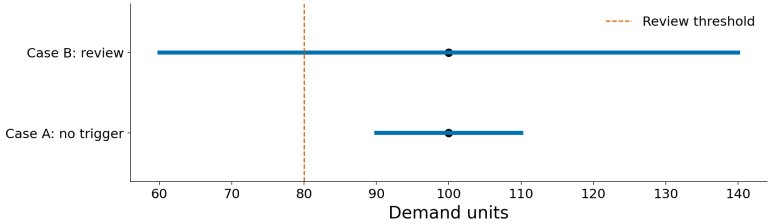



Coding Note, Example 5.20.

# Connect a threshold signal to a review procedure

A threshold rule maps a forecast feature to a specified response.

1. Both point forecasts are 100; lower bounds are 90 and 60.
2. Prespecify review when the lower bound is below 80.
3. Only the second triggers; determine action quantity with costs and constraints.

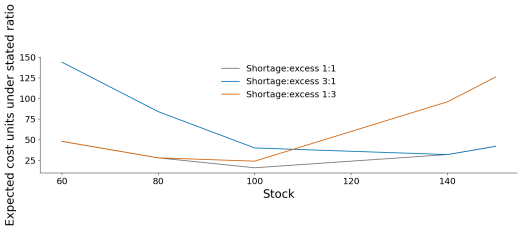



Coding Note, Example 5.21.

# Stress the cost assumptions before recommending

Sensitivity analysis changes stated assumptions while holding the comparison conditions fixed.

1. Keep demand outcomes and probabilities unchanged.
2. Recalculate for cost ratios 1:1,3:1,1:3.
3. The corresponding grid choices are 100,140,100.

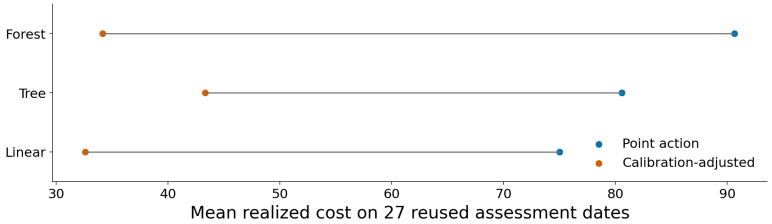



Coding Note, Example 5.22.

# Evaluate a calibration-error action adjustment

A conditional quantile is a cutoff given known inputs. A common residual offset is an approximation.

1. Fit fixed Linear,Tree,Forest on training only.
2. Estimate each .75 error offset using calibration only.
3. Teaching replay: tree settings were selected using this calibration block in Lecture 4.

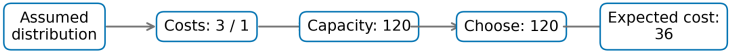



Coding Note, Example 5.23.

# Write a recommendation that can be audited

An auditable recommendation records assumptions, feasible actions, cost arithmetic and a review condition.

1. Assumed mean demand 108; median 100; costs 3:1.
2. Capacity 120 binds the unconstrained 140-unit action.
3. Recommend feasible stock 120; conditional expected cost 36.

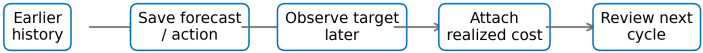



Coding Note, Example 5.24.

# Replay decisions before attaching later outcomes

Save what was known and chosen before attaching the later outcome.

1. Save prior origin, target date, point, bounds and chosen stock.
2. Apply the stated nonnegative stock and capacity-400 limits.
3. Then attach demand and cost; export all 27 decision records.




Coding Note, Example 5.25.

# What would make you change the approach?

Give a specific piece of evidence and the response it would trigger.

1. Unfamiliar input combinations or a changed relationship.
2. Changed shortage/excess costs or binding capacity.
3. Measured errors, coverage or decision loss no longer fit the intended use.

# Use the complete code; submit separate homework

The same example IDs connect the notes, slides, classroom code and homework references.

1. Classroom examples 5.1–5.25 contain complete worked code and figures.
2. Assigned homework supplies prompts, starter code, hints and references to Coding Note examples.
3. Lecture 6 extends these records to repeated forecast origins and integrated evaluation.